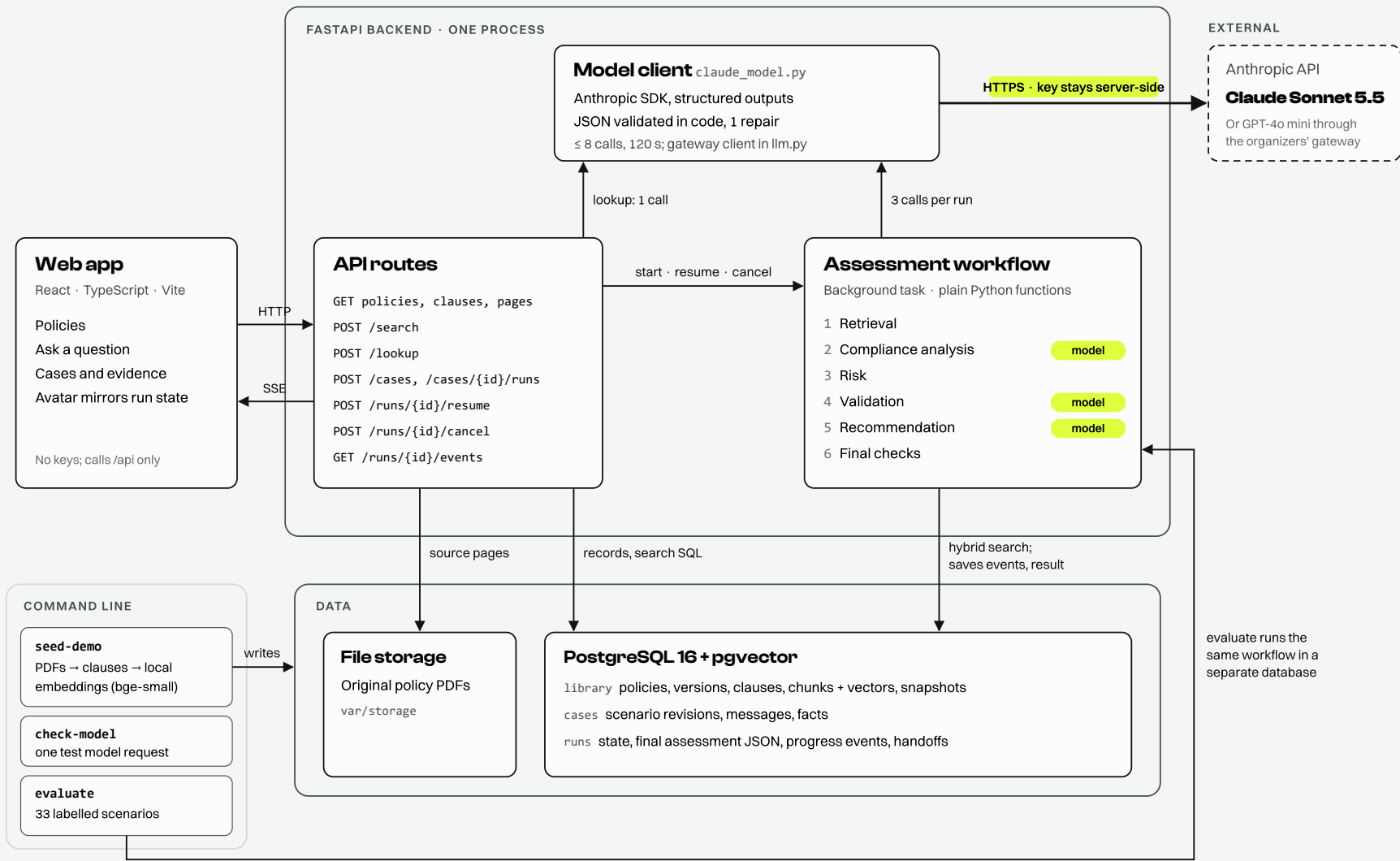
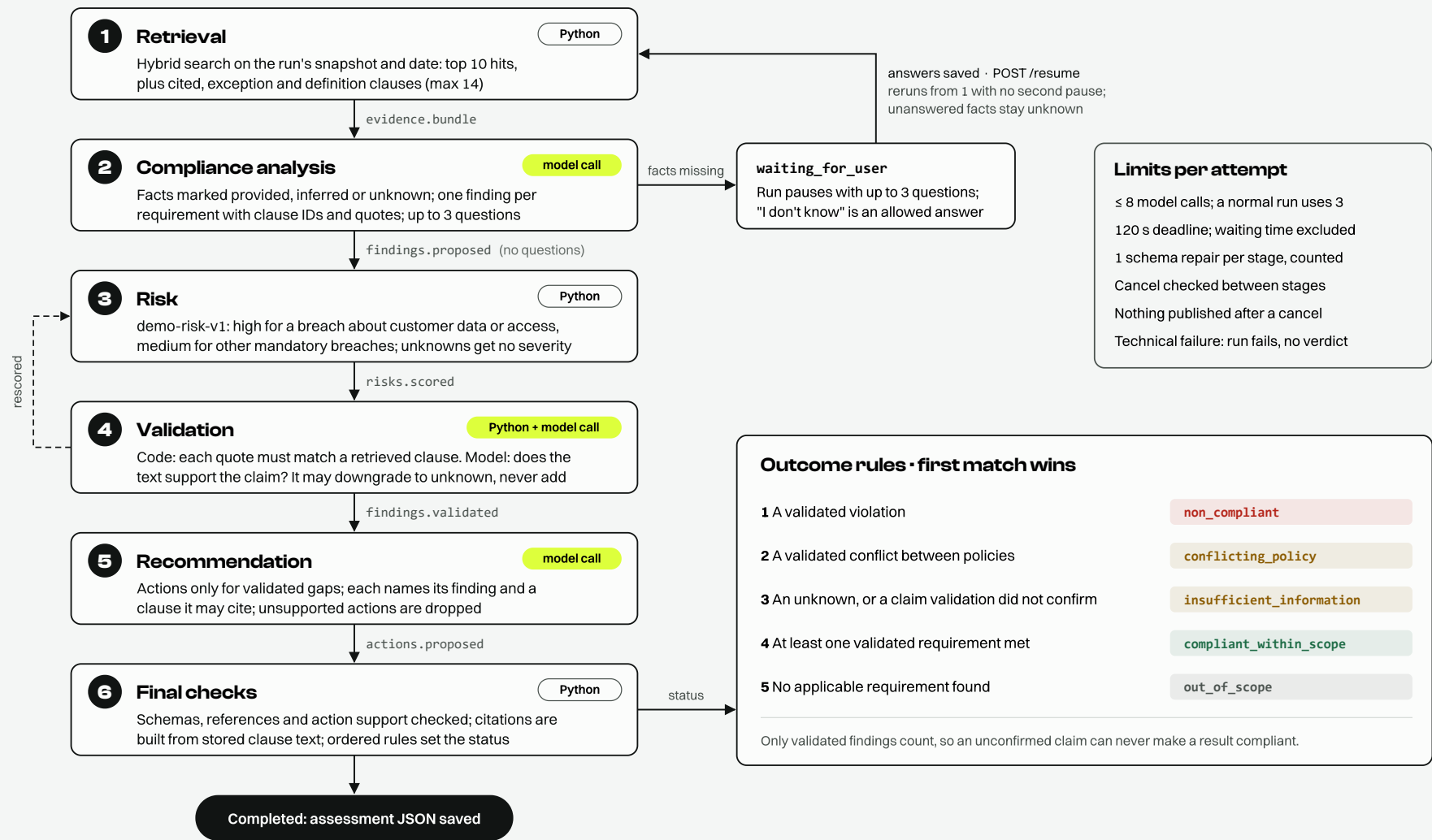




The browser talks only to FastAPI: JSON over HTTP for actions, Server-Sent Events for run progress. One API process holds the routes, the assessment workflow (a background task, so the API can return a run ID at once) and a single model client. Search runs inside PostgreSQL (full-text ranking plus pgvector, fused by reciprocal rank); query and clause embeddings are computed locally, so search needs no key. The only runtime call that leaves the machine is the model client's HTTPS request to the Anthropic API for Claude Sonnet 5.5 (dashed: external), whose replies are constrained to each stage's JSON schema. The organizers' GPT-4o mini gateway is a supported alternative; it caps replies at 500 tokens, so there a cut-off reply is continued and joined before it is validated. Restarting the API fails an in-flight run; completed results stay saved.

Clause — assessment workflow

services/api/app/workflow/ · orchestrator.py runs the stages in order
Five specialist roles, one backend, one model configuration



Each numbered step is one role with its own function and output schema. After every step the run saves a progress event (streamed to the browser over SSE) and a handoff record naming the sender, receiver and message type shown on the arrows; the trace view reads those records. Retrieval and risk are deterministic code; analysis, the validation support check and recommendation are one model call each. If retrieval finds no clauses, analysis is skipped and the result is out of scope. These are application-level handoffs inside one process, not the standardized Agent2Agent protocol.